

01_sol: Tool-Using Support Agent

Contains:

- the same scenario as 01_mock
- one complete reference implementation
- grading tests

```
In [ ]: # Chunk overview: Prepare imports, fixtures, and helper scaffolding used by
# Why this chunk exists: it makes the solution easier to reason about under
# Import required modules for this solution step.
import inspect
# Import required modules for this solution step.
import json
# Import required modules for this solution step.
from copy import deepcopy
# Import required modules for this solution step.
from typing import Any, Callable

# Assign computed data to a named variable for later use.
ORDERS_DB = {
    # Execute this line as part of the solution flow.
    "u-100": [
        # Execute this line as part of the solution flow.
        {"order_id": "o-900", "status": "delivered", "days_since_delivery":
# Execute this line as part of the solution flow.
        ],
        # Execute this line as part of the solution flow.
        "u-200": [
            # Execute this line as part of the solution flow.
            {"order_id": "o-901", "status": "in_transit", "days_since_delivery":
# Execute this line as part of the solution flow.
        ],
# Execute this line as part of the solution flow.
    }
# Assign computed data to a named variable for later use.
REFUNDS: list[dict[str, Any]] = []

# Define `reset_state` so this step is reusable and testable.
def reset_state() -> None:
    # Call this function to perform the next operation.
    REFUNDS.clear()

# Define `get_orders` so this step is reusable and testable.
def get_orders(user_id: str) -> list[dict[str, Any]]:
    # Check this condition to choose the correct branch.
    if user_id == "boom":
        # Raise explicit error to fail fast on invalid state.
        raise RuntimeError("orders backend unavailable")
```

```

# Return the computed value for the caller.
return deepcopy(ORDERS_DB.get(user_id, []))

# Define `policy_check` so this step is reusable and testable.
def policy_check(order_id: str, reason: str, days_since_delivery: int) -> dict[str, Any]:
    # Execute this line as part of the solution flow.
    eligible = reason.lower() in {"damaged", "wrong_item"} and days_since_delivery < 30
    # Return the computed value for the caller.
    return {
        # Execute this line as part of the solution flow.
        "order_id": order_id,
        # Execute this line as part of the solution flow.
        "eligible": eligible,
        # Execute this line as part of the solution flow.
        "policy_reason": "allowed" if eligible else "outside_policy",
    }
    # Execute this line as part of the solution flow.

# Define `create_refund` so this step is reusable and testable.
def create_refund(order_id: str, amount: float) -> dict[str, Any]:
    # Check this condition to choose the correct branch.
    if amount <= 0:
        # Raise explicit error to fail fast on invalid state.
        raise ValueError("refund amount must be positive")
    # Assign computed data to a named variable for later use.
    record = {"order_id": order_id, "amount": amount, "status": "submitted"}
    # Call this function to perform the next operation.
    REFUNDS.append(record)
    # Return the computed value for the caller.
    return deepcopy(record)

# Assign computed data to a named variable for later use.
TOOL_REGISTRY: dict[str, Callable[..., Any]] = {
    # Execute this line as part of the solution flow.
    "get_orders": get_orders,
    # Execute this line as part of the solution flow.
    "policy_check": policy_check,
    # Execute this line as part of the solution flow.
    "create_refund": create_refund,
}
# Execute this line as part of the solution flow.

# Define class `ScriptedModel` to organize related behavior.
class ScriptedModel:
    # Define `__init__` so this step is reusable and testable.
    def __init__(self, responses: list[dict[str, Any]]) -> None:
        # Assign computed data to a named variable for later use.
        self._responses = deepcopy(responses)
        # Assign computed data to a named variable for later use.
        self._index = 0
        # Assign computed data to a named variable for later use.
        self.last_system_prompt: str | None = None

```

```

        # Assign computed data to a named variable for later use.
        self.last_tools: list[dict[str, Any]] = []

    # Define `__call__` so this step is reusable and testable.
    def __call__(self, messages: list[dict[str, Any]], system_prompt: str, tools: list[dict[str, Any]]) -> dict[str, Any]:
        # Assign computed data to a named variable for later use.
        self.last_system_prompt = system_prompt
        # Assign computed data to a named variable for later use.
        self.last_tools = deepcopy(tools)
        # Check this condition to choose the correct branch.
        if self._index >= len(self._responses):
            # Return the computed value for the caller.
            return {
                # Execute this line as part of the solution flow.
                "stop_reason": "end_turn",
                # Execute this line as part of the solution flow.
                "content": [{"type": "text", "text": "No scripted response for this step."}],
                # Execute this line as part of the solution flow.
            }
        # Assign computed data to a named variable for later use.
        response = self._responses[self._index]
        # Assign computed data to a named variable for later use.
        self._index += 1
        # Return the computed value for the caller.
        return deepcopy(response)

```

```

In [ ]: # Chunk overview: Implement the final reference solution in a clean, stepwise manner.
        # Why this chunk exists: it makes the solution easier to reason about under uncertainty.
        # Define `build_agent_system_prompt` so this step is reusable and testable.
        def build_agent_system_prompt() -> str:
            # Return the computed value for the caller.
            return (
                # Execute this line as part of the solution flow.
                "You are a customer support agent. ",
                # Execute this line as part of the solution flow.
                "Use tools to verify order and policy facts before answering. ",
                # Execute this line as part of the solution flow.
                "Never fabricate tool outputs or policy decisions. ",
                # Execute this line as part of the solution flow.
                "If required inputs are missing, ask a concise clarifying question. ",
                # Execute this line as part of the solution flow.
                "Keep final answers short and grounded in tool results."
            )
        # Call this function to perform the next operation.

    # Define `build_tool_schemas` so this step is reusable and testable.
    def build_tool_schemas(tool_registry: dict[str, Callable[..., Any]]) -> list[dict[str, Any]]:
        # Assign computed data to a named variable for later use.
        schemas: list[dict[str, Any]] = []
        # Iterate through items to process each element deterministically.
        for name, fn in sorted(tool_registry.items()):
            # Assign computed data to a named variable for later use.
            sig = inspect.signature(fn)
            # Assign computed data to a named variable for later use.
            properties: dict[str, dict[str, str]] = {}

```

```

# Assign computed data to a named variable for later use.
required: list[str] = []
# Iterate through items to process each element deterministically.
for param in sig.parameters.values():
    # Assign computed data to a named variable for later use.
    properties[param.name] = {"type": "string"}
    # Check this condition to choose the correct branch.
    if param.default is inspect._empty:
        # Call this function to perform the next operation.
        required.append(param.name)
# Execute this line as part of the solution flow.
schemas.append(
    # Execute this line as part of the solution flow.
    {
        # Execute this line as part of the solution flow.
        "name": name,
        # Execute this line as part of the solution flow.
        "description": f"Call {name} to retrieve deterministic backed",
        # Execute this line as part of the solution flow.
        "input_schema": {
            # Execute this line as part of the solution flow.
            "type": "object",
            # Execute this line as part of the solution flow.
            "properties": properties,
            # Execute this line as part of the solution flow.
            "required": required,
            # Execute this line as part of the solution flow.
            "additionalProperties": True,
        },
        # Execute this line as part of the solution flow.
    },
    # Execute this line as part of the solution flow.
)
# Call this function to perform the next operation.
)
# Return the computed value for the caller.
return schemas

# Define `validate_tool_call` so this step is reusable and testable.
def validate_tool_call(tool_call: dict[str, Any], tool_registry: dict[str, Callable]) >
    # Assign computed data to a named variable for later use.
    required = {"id", "name", "input"}
    # Check this condition to choose the correct branch.
    if not required.issubset(tool_call):
        # Return the computed value for the caller.
        return "tool_call_missing_required_fields"

    # Assign computed data to a named variable for later use.
    name = tool_call["name"]
    # Assign computed data to a named variable for later use.
    payload = tool_call["input"]
    # Check this condition to choose the correct branch.
    if name not in tool_registry:
        # Return the computed value for the caller.
        return "unknown_tool"
    # Check this condition to choose the correct branch.

```

```

if not isinstance(payload, dict):
    # Return the computed value for the caller.
    return "tool_input_must_be_object"

# Assign computed data to a named variable for later use.
sig = inspect.signature(tool_registry[name])
# Assign computed data to a named variable for later use.
missing: list[str] = []
# Iterate through items to process each element deterministically.
for param in sig.parameters.values():
    # Check this condition to choose the correct branch.
    if param.default is inspect._empty and param.name not in payload:
        # Call this function to perform the next operation.
        missing.append(param.name)
# Check this condition to choose the correct branch.
if missing:
    # Return the computed value for the caller.
    return f"missing_required_args:{','.join(sorted(missing))}"

# Return the computed value for the caller.
return None

# Define `execute_tool_call` so this step is reusable and testable.
def execute_tool_call(tool_call: dict[str, Any], tool_registry: dict[str, Callable]) -> dict[str, Any]:
    # Assign computed data to a named variable for later use.
    tool_id = str(tool_call.get("id", "missing_id"))
    # Assign computed data to a named variable for later use.
    tool_name = str(tool_call.get("name", "missing_name"))

    # Assign computed data to a named variable for later use.
    validation_error = validate_tool_call(tool_call, tool_registry)
    # Check this condition to choose the correct branch.
    if validation_error:
        # Return the computed value for the caller.
        return {
            # Execute this line as part of the solution flow.
            "type": "tool_result",
            # Execute this line as part of the solution flow.
            "tool_use_id": tool_id,
            # Execute this line as part of the solution flow.
            "is_error": True,
            # Assign computed data to a named variable for later use.
            "content": json.dumps({"error": validation_error}, sort_keys=True)
        }
    # Execute this line as part of the solution flow.

# Start guarded block to handle potential runtime errors.
try:
    # Assign computed data to a named variable for later use.
    result = tool_registry[tool_name](**tool_call["input"])
    # Return the computed value for the caller.
    return {
        # Execute this line as part of the solution flow.
        "type": "tool_result",
        # Execute this line as part of the solution flow.

```

```

        "tool_use_id": tool_id,
        # Execute this line as part of the solution flow.
        "is_error": False,
        # Assign computed data to a named variable for later use.
        "content": json.dumps({"result": result}, sort_keys=True),
        # Execute this line as part of the solution flow.
    }
    # Handle expected failure path and keep behavior predictable.
    except Exception as exc: # pragma: no cover – explicit for interview rc
        # Return the computed value for the caller.
        return {
            # Execute this line as part of the solution flow.
            "type": "tool_result",
            # Execute this line as part of the solution flow.
            "tool_use_id": tool_id,
            # Execute this line as part of the solution flow.
            "is_error": True,
            # Assign computed data to a named variable for later use.
            "content": json.dumps({"error": str(exc)}, sort_keys=True),
            # Execute this line as part of the solution flow.
        }

# Define `run_agent` so this step is reusable and testable.
def run_agent(
    # Execute this line as part of the solution flow.
    user_prompt: str,
    # Execute this line as part of the solution flow.
    model: Callable[[list[dict[str, Any]], str, list[dict[str, Any]]], dict],
    # Execute this line as part of the solution flow.
    tool_registry: dict[str, Callable[..., Any]],
    # Assign computed data to a named variable for later use.
    max_steps: int = 6,
    # Execute this line as part of the solution flow.
) -> dict[str, Any]:
    # Assign computed data to a named variable for later use.
    messages: list[dict[str, Any]] = [{"role": "user", "content": [{"type":
    # Assign computed data to a named variable for later use.
    system_prompt = build_agent_system_prompt()
    # Assign computed data to a named variable for later use.
    tools = build_tool_schemas(tool_registry)

    # Iterate through items to process each element deterministically.
    for _ in range(max_steps):
        # Assign computed data to a named variable for later use.
        response = model(messages, system_prompt, tools)
        # Assign computed data to a named variable for later use.
        stop_reason = response.get("stop_reason")
        # Assign computed data to a named variable for later use.
        content = response.get("content", [])
        # Check this condition to choose the correct branch.
        if not isinstance(content, list):
            # Raise explicit error to fail fast on invalid state.
            raise RuntimeError("assistant_content_must_be_list")
        # Call this function to perform the next operation.
        messages.append({"role": "assistant", "content": content})

```

```

# Check this condition to choose the correct branch.
if stop_reason == "tool_use":
    # Execute this line as part of the solution flow.
    tool_calls = [block for block in content if block.get("type") == "tool_use"]
    # Check this condition to choose the correct branch.
    if not tool_calls:
        # Raise explicit error to fail fast on invalid state.
        raise RuntimeError("tool_use_without_blocks")
    # Assign computed data to a named variable for later use.
    tool_results: list[dict[str, Any]] = []
    # Iterate through items to process each element deterministically.
    for tool_call in tool_calls:
        # Call this function to perform the next operation.
        tool_results.append(execute_tool_call(tool_call, tool_registry))
    # Call this function to perform the next operation.
    messages.append({"role": "user", "content": tool_results})
    # Execute this line as part of the solution flow.
    continue

# Check this condition to choose the correct branch.
if stop_reason == "end_turn":
    # Assign computed data to a named variable for later use.
    text = " ".join(
        # Call this function to perform the next operation.
        block.get("text", "").strip()
        # Iterate through items to process each element deterministically.
        for block in content
        # Check this condition to choose the correct branch.
        if block.get("type") == "text"
    )
    # Call this function to perform the next operation.
    ).strip()
    # Return the computed value for the caller.
    return {"final_text": text, "messages": messages}

# Raise explicit error to fail fast on invalid state.
raise RuntimeError(f"unsupported_stop_reason:{stop_reason}")

# Raise explicit error to fail fast on invalid state.
raise RuntimeError("max_steps_exceeded")

```

```

In [ ]: # Chunk overview: Run checks that prove the implementation meets the problem
# Why this chunk exists: it makes the solution easier to reason about under
# Define `_tool_result_blocks` so this step is reusable and testable.
def _tool_result_blocks(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    # Assign computed data to a named variable for later use.
    blocks: list[dict[str, Any]] = []
    # Iterate through items to process each element deterministically.
    for message in messages:
        # Check this condition to choose the correct branch.
        if message.get("role") != "user":
            # Execute this line as part of the solution flow.
            continue
    # Iterate through items to process each element deterministically.
    for block in message.get("content", []):
        # Check this condition to choose the correct branch.

```



```

        if block.get("type") == "tool_result":
            # Call this function to perform the next operation.
            blocks.append(block)
        # Return the computed value for the caller.
        return blocks

# Define `run_exam01_tests` so this step is reusable and testable.
def run_exam01_tests() -> None:
    # Call this function to perform the next operation.
    reset_state()
    # Assign computed data to a named variable for later use.
    prompt = build_agent_system_prompt().lower()
    # Assert expected behavior to validate correctness.
    assert "use tools" in prompt
    # Assert expected behavior to validate correctness.
    assert "never fabricate" in prompt
    # Assert expected behavior to validate correctness.
    assert "clarifying question" in prompt
    # Assign computed data to a named variable for later use.
    tool_defs = build_tool_schemas(TOOL_REGISTRY)
    # Assert expected behavior to validate correctness.
    assert sorted(tool["name"] for tool in tool_defs) == sorted(TOOL_REGISTRY)

    # Existing inline note.
    # 1) Single tool call
    # Assign computed data to a named variable for later use.
    model = ScriptedModel(
        # Execute this line as part of the solution flow.
        [
            # Execute this line as part of the solution flow.
            {
                # Execute this line as part of the solution flow.
                "stop_reason": "tool_use",
                # Execute this line as part of the solution flow.
                "content": [
                    # Execute this line as part of the solution flow.
                    {"type": "tool_use", "id": "t1", "name": "get_orders", "
                    # Execute this line as part of the solution flow.
                    },
                # Execute this line as part of the solution flow.
            ],
            # Execute this line as part of the solution flow.
            {"stop_reason": "end_turn", "content": [{"type": "text", "text":
        # Execute this line as part of the solution flow.
    ]
    # Call this function to perform the next operation.
    )
    # Assign computed data to a named variable for later use.
    result = run_agent("Where is my order?", model, TOOL_REGISTRY)
    # Assert expected behavior to validate correctness.
    assert "delivered" in result["final_text"].lower()
    # Assert expected behavior to validate correctness.
    assert len(_tool_result_blocks(result["messages"])) == 1
    # Assert expected behavior to validate correctness.
    assert model.last_system_prompt is not None

```



```

# Assert expected behavior to validate correctness.
assert "use tools" in model.last_system_prompt.lower()
# Assert expected behavior to validate correctness.
assert sorted(tool["name"] for tool in model.last_tools) == sorted(TOOL_

# Existing inline note.
# 2) Multiple tools in one model turn
# Assign computed data to a named variable for later use.
model = ScriptedModel(
    # Execute this line as part of the solution flow.
    [
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "stop_reason": "tool_use",
            # Execute this line as part of the solution flow.
            "content": [
                # Execute this line as part of the solution flow.
                {
                    # Execute this line as part of the solution flow.
                    "type": "tool_use",
                    # Execute this line as part of the solution flow.
                    "id": "t2",
                    # Execute this line as part of the solution flow.
                    "name": "policy_check",
                    # Execute this line as part of the solution flow.
                    "input": {"order_id": "o-900", "reason": "damaged",
# Execute this line as part of the solution flow.
},
# Execute this line as part of the solution flow.
{
    # Execute this line as part of the solution flow.
    "type": "tool_use",
    # Execute this line as part of the solution flow.
    "id": "t3",
    # Execute this line as part of the solution flow.
    "name": "create_refund",
    # Execute this line as part of the solution flow.
    "input": {"order_id": "o-900", "amount": 42.5},
    # Execute this line as part of the solution flow.
},
        # Execute this line as part of the solution flow.
    ],
    # Execute this line as part of the solution flow.
},
    # Execute this line as part of the solution flow.
    {"stop_reason": "end_turn", "content": [{"type": "text", "text":
# Execute this line as part of the solution flow.
    ]
}
# Call this function to perform the next operation.
)
# Assign computed data to a named variable for later use.
result = run_agent("Refund my damaged item", model, TOOL_REGISTRY)
# Assert expected behavior to validate correctness.
assert len(_tool_result_blocks(result["messages"])) == 2
# Assert expected behavior to validate correctness.

```

```

assert REFUNDS and REFUNDS[-1]["order_id"] == "o-900"
# Call this function to perform the next operation.
assistant_idx = next(i for i, m in enumerate(result["messages"])) if m["r
# Assert expected behavior to validate correctness.
assert result["messages"][assistant_idx + 1]["role"] == "user"
# Assert expected behavior to validate correctness.
assert all(
    # Execute this line as part of the solution flow.
    block.get("type") == "tool_result" for block in result["messages"][a
# Call this function to perform the next operation.
)

# Existing inline note.
# 3) Missing args -> is_error tool message
# Assign computed data to a named variable for later use.
model = ScriptedModel(
    # Execute this line as part of the solution flow.
    [
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "stop_reason": "tool_use",
            # Execute this line as part of the solution flow.
            "content": [{"type": "tool_use", "id": "bad-args", "name": "
# Execute this line as part of the solution flow.
        },
        # Execute this line as part of the solution flow.
        {"stop_reason": "end_turn", "content": [{"type": "text", "text":
# Execute this line as part of the solution flow.
    ]
# Call this function to perform the next operation.
)
# Assign computed data to a named variable for later use.
result = run_agent("debug", model, TOOL_REGISTRY)
# Assign computed data to a named variable for later use.
tool_block = _tool_result_blocks(result["messages"])[0]
# Assert expected behavior to validate correctness.
assert tool_block["is_error"] is True
# Assert expected behavior to validate correctness.
assert "missing_required_args" in tool_block["content"]

# Existing inline note.
# 4) Runtime exception -> is_error tool message
# Assign computed data to a named variable for later use.
model = ScriptedModel(
    # Execute this line as part of the solution flow.
    [
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "stop_reason": "tool_use",
            # Execute this line as part of the solution flow.
            "content": [
                # Execute this line as part of the solution flow.
                {"type": "tool_use", "id": "boom", "name": "get_orders",
            # Execute this line as part of the solution flow.

```

```

        ],
        # Execute this line as part of the solution flow.
    },
    # Execute this line as part of the solution flow.
    {"stop_reason": "end_turn", "content": [{"type": "text", "text":
# Execute this line as part of the solution flow.
    ]
# Call this function to perform the next operation.
)
# Assign computed data to a named variable for later use.
result = run_agent("debug", model, TOOL_REGISTRY)
# Assign computed data to a named variable for later use.
tool_block = _tool_result_blocks(result["messages"])[0]
# Assert expected behavior to validate correctness.
assert tool_block["is_error"] is True
# Assert expected behavior to validate correctness.
assert "backend unavailable" in tool_block["content"]

# Existing inline note.
# 5) Max step protection
# Assign computed data to a named variable for later use.
model = ScriptedModel(
    # Execute this line as part of the solution flow.
    [
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "stop_reason": "tool_use",
            # Execute this line as part of the solution flow.
            "content": [{"type": "tool_use", "id": "loop1", "name": "get
# Execute this line as part of the solution flow.
        },
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "stop_reason": "tool_use",
            # Execute this line as part of the solution flow.
            "content": [{"type": "tool_use", "id": "loop2", "name": "get
# Execute this line as part of the solution flow.
        },
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "stop_reason": "tool_use",
            # Execute this line as part of the solution flow.
            "content": [{"type": "tool_use", "id": "loop3", "name": "get
# Execute this line as part of the solution flow.
        },
        # Execute this line as part of the solution flow.
    ]
# Call this function to perform the next operation.
)
# Start guarded block to handle potential runtime errors.
try:
    # Assign computed data to a named variable for later use.
    run_agent("loop", model, TOOL_REGISTRY, max_steps=2)

```

```
        # Raise explicit error to fail fast on invalid state.
        raise AssertionError("Expected max_steps_exceeded")
    # Handle expected failure path and keep behavior predictable.
except RuntimeError as exc:
    # Assert expected behavior to validate correctness.
    assert "max_steps_exceeded" in str(exc)

    # Call this function to perform the next operation.
    print("01_mock tests passed")

# Call this function to perform the next operation.
run_exam01_tests()
```